Nombre:	**Matrícula:**
Ángel Adrián Velez Contreras	2145646
Alonso Morales Quintanilla	2145757
Jairany López Ramos	2145407

Nombre del curso: Componentes en Lógica de Negocios	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Unidad: 4	**Actividad:** Actividad Fundamental 4 **Link del repositorio:** https://github.com/jyrnso0/Unidad-4
Fecha: 28/10/2026	

Bibliografia (Formato APA)

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Evans, E., & Fowler, M. (2004). Specifications. https://martinfowler.com/apsupp/spec.pdf

Ramírez, S. (s. f.). FastAPI documentation. https://fastapi.tiangolo.com

Python Software Foundation. (s. f.). decimal: Decimal fixed point and floating point arithmetic. https://docs.python.org/3/library/decimal.html

Gómez Marroquín, M. A. (2026). CLN_U4 Manual de alumnos ilustrado y Práctica guiada TechStore [Material de clase]. FIME, UANL.

Objetivo:

Desarrollar el núcleo lógico para la gestión de pedidos de TechStore implementando una arquitectura basada en el Diseño Orientado al Dominio (Domain-Driven Design). El proyecto busca modelar entidades, agregados y máquinas de estado deterministas que garanticen el cumplimiento de invariantes de negocio estrictas y reglas de descuento compuestas mediante el patrón Specification. Asimismo, se tiene como meta exponer el modelo de dominio a través de una API REST con FastAPI, validando su comportamiento de forma integral mediante 42 pruebas automatizadas en memoria, asegurando que el sistema trabaje de manera independiente a infraestructuras de persistencia externas.

1. Introducción y Arquitectura del Dominio

El presente proyecto desarrolla el núcleo de lógica de negocio para la gestión de pedidos de TechStore, aplicando una arquitectura limpia orientada al dominio (Domain-Driven Design). Se modelan entidades inmutables y de agregado, se garantiza la consistencia transaccional mediante invariantes estrictas, se encapsulan las reglas de descuento utilizando el patrón de diseño Specification con soporte para operadores booleanos (&, |, ~), se implementa una máquina de estados determinista gobernada por eventos de dominio (Domain Events), y se expone el sistema mediante una API REST en FastAPI validada por pruebas automatizadas con pytest.

El sistema trabaja enteramente en memoria con datos cargados a partir de fixtures reproducibles en formato CSV (clientes, productos, pedidos, detalle de pedidos y escenarios), desacoplando por completo las reglas de negocio de mecanismos de persistencia o frameworks externos.

2. Cálculo Manual de Pedidos

Antes de iniciar la codificación, se inspeccionaron los archivos pedidos.csv, detalle_pedidos.csv y productos.csv para realizar el cálculo manual de unidades e importe de los pedidos muestra solicitados:

Pedido	Cliente	Unidades	Total Manual (MXN)
O001	C001 (Ana Torres)	4 unidades (2 SKU001 + 2 SKU002)	$2,100.00 MXN (2×$600.00 + 2×$450.00)
O006	C006 (Pedro Soto)	1 unidad (1 SKU008)	$18,000.00 MXN (1×$18,000.00)
O010	C010 (Jorge Luna)	2 unidades (1 SKU003 + 1 SKU009)	$500.00 MXN (1×$120.00 + 1×$380.00)

3. Punto de Control: Carga de Datos desde CSV (cargar_csv.py)

Al ejecutar el script de carga cargar_csv.py una vez implementadas las entidades y Value Objects en app/domain.py, se comprueba que los 16 pedidos del fixture son cargados en memoria en estado DRAFT, con sus respectivas unidades y montos calculados exactamente:

```
$ python cargar_csv.py
O001 DRAFT 4 2100.00
O002 DRAFT 2 1050.00
O003 DRAFT 4 2100.00
O004 DRAFT 1 850.00
O005 DRAFT 4 2100.00
O006 DRAFT 1 18000.00
O007 DRAFT 0 0
O008 DRAFT 20 2400.00
O009 DRAFT 19 2280.00
O010 DRAFT 2 500.00
O011 DRAFT 3 1500.00
O012 DRAFT 1 850.00
O013 DRAFT 1 600.00
O014 DRAFT 1 600.00
O015 DRAFT 1 450.00
O016 DRAFT 1 2100.00
```

4. Bitácora Resuelta de los 15 Escenarios (E01 a E15)

Se ejecutaron los 15 escenarios contenidos en datos/escenarios.csv sobre instancias limpias de load_fixture(). A continuación se detalla la bitácora técnica con estado inicial, operación, resultado real, eventos y justificación de negocio:

Caso	Pedido	Estado Ini.	Operación	Resultado Real / Eventos	Explicación de Negocio
E01	O001	DRAFT	evaluar_descuento	10% (Sin eventos)	Cliente VIP (C001), total $2100.00 > 500, 4 unidades >= 4, sin promoción. Cumple regla del 10%.
E02	O002	DRAFT	evaluar_descuento	0% (Sin eventos)	Cliente no VIP (C002) y total $1050.00 <= 5000. No cumple regla del 10% ni del 5%.
E03	O003	DRAFT	confirmar	RECHAZO (InvalidOrderData) (Ningún evento)	Cliente C003 está inactivo (activo=0). La invariante exige cliente activo para confirmar.
E04	O005	DRAFT	evaluar_descuento	0% (Sin eventos)	El pedido tiene promoción activa (tiene_promocion=1). La especificación ~ActivePromotionSpec lo excluye.
E05	O006	DRAFT	evaluar_descuento	5% (Sin eventos)	Cliente no VIP, pero total $18000.00 > 5000 y sin promoción. Aplica la regla alternativa del 5%.
E06	O007	DRAFT	confirmar	RECHAZO (InvalidOrderData) (Ningún evento)	Pedido sin artículos (0 unidades). La regla exige al menos un artículo para confirmar.
E07	O008	DRAFT	confirmar	ÉXITO -> CONFIRMED ['OrderConfirmed']	Tiene exactamente 20 unidades de SKU003 y cliente activo. Es el límite máximo válido.
E08	O009	DRAFT	agregar SKU003 x2	RECHAZO (OrderItemLimitExceeded) (Ningún evento)	El pedido ya contiene 19 unidades. Agregar 2 daría 21, excediendo el límite estricto de 20.
E09	O010	DRAFT	evaluar_descuento	0% (Sin eventos)	Total es exactamente $500.00 MXN. La especificación exige total

					estrictamente mayor a 500 (>500).
E10	O011	DRAFT	evaluar_descuento	0% (Sin eventos)	Cliente VIP y total $1500 > 500, pero solo tiene 3 unidades. La regla exige al menos 4 unidades (>=4).
E11	O012	DRAFT	confirmar y enviar sin guía	RECHAZO (InvalidOrderData) ['OrderConfirmed']	Pasa a CONFIRMED emitiendo OrderConfirmed. El envío falla porque la guía no puede ser vacía.
E12	O013	DRAFT	confirmar dos veces	RECHAZO (InvalidOrderTransition) ['OrderConfirmed']	Primera confirmación exitosa. La segunda se rechaza porque solo pedidos en DRAFT pueden confirmarse.
E13	O014	DRAFT	confirmar, enviar y cancelar	RECHAZO (InvalidOrderTransition) ['OrderConfirmed', 'OrderShipped']	Pasa a CONFIRMED y SHIPPED. La cancelación falla porque solo pedidos en CONFIRMED pueden cancelarse.
E14	O015	DRAFT	confirmar, enviar y devolver	ÉXITO -> RETURNED ['OrderConfirmed', 'OrderShipped', 'OrderReturned']	Ciclo válido: DRAFT -> CONFIRMED -> SHIPPED -> RETURNED. Emite los 3 eventos correspondientes.
E15	O016	DRAFT	confirmar, enviar y entregar	ÉXITO -> DELIVERED ['OrderConfirmed', 'OrderShipped', 'OrderDelivered']	Ciclo válido: DRAFT -> CONFIRMED -> SHIPPED -> DELIVERED. Emite los 3 eventos correspondientes.

5. Máquina de Estados y Explicación de Domain Events

El ciclo de vida del agregado Order sigue una máquina de estados finita determinista. Todo pedido nace en DRAFT y solo puede avanzar por las transiciones del diagrama: DRAFT → CONFIRMED con confirm(), CONFIRMED → SHIPPED con ship(), CONFIRMED → CANCELLED con cancel(), SHIPPED → DELIVERED con deliver() y SHIPPED → RETURNED con return_order(). DELIVERED, CANCELLED y RETURNED son estados finales. Cualquier operación fuera de este grafo lanza InvalidOrderTransition, y si los datos no cumplen una regla, InvalidOrderData. En ambos casos el pedido conserva su estado y no se registra ningún evento. Por ejemplo, un pedido en DRAFT sin artículos o con cliente inactivo no se confirma y permanece en DRAFT.



Explicación de los Domain Events:

- **OrderConfirmed:** se emite al pasar de DRAFT a CONFIRMED, después de validar que el pedido tiene al menos un artículo y que el cliente está activo.
- **OrderShipped:** se emite al pasar de CONFIRMED a SHIPPED, después de validar que se proporcionó un número de guía no vacío.
- **OrderCancelled:** se emite al pasar de CONFIRMED a CANCELLED, después de validar que se indicó un motivo no vacío. Si el pedido ya fue enviado (SHIPPED) o sigue en DRAFT, la cancelación se rechaza.
- **OrderDelivered:** se emite al pasar de SHIPPED a DELIVERED cuando el pedido enviado llega al cliente.
- **OrderReturned:** se emite al pasar de SHIPPED a RETURNED cuando el pedido

enviado es devuelto.

Principio de atomicidad de eventos: cada método valida primero las reglas de datos y de transición, después cambia el estado y, por último, agrega el evento a order.events. Si se produce una excepción (InvalidOrderData o InvalidOrderTransition), la ejecución se detiene antes de modificar el estado y de registrar el evento. Así el historial de eventos refleja con exactitud lo que realmente ocurrió.

6. Patrón Specification y Reglas Compuestas

En lugar de dispersar condicionales en el agregado o en los controladores web, se utilizó el patrón Specification para encapsular cada criterio de evaluación en clases reutilizables y componibles mediante sobrecarga de operadores:

• MinimumAmountSpec(Decimal('500')): Valida que order.total.amount > 500 (umbral estricto).
• VIPCustomerSpec(): Valida que order.customer.is_vip sea True.
• MinimumUnitsSpec(4): Valida que order.units >= 4.
• ActivePromotionSpec(): Valida si order.has_promotion es True.

Composición de Reglas:
1. Regla del 10% = MinimumAmountSpec(500) & VIPCustomerSpec() & MinimumUnitsSpec(4) & ~ActivePromotionSpec()
2. Regla del 5% = MinimumAmountSpec(5000) & ~ActivePromotionSpec()

Prioridad de evaluación: Si ambas reglas se cumplen simultáneamente, el sistema asigna prioritariamente el 10% de descuento.

7. API REST con FastAPI y Respuestas HTTP

La capa de infraestructura expone la API REST utilizando FastAPI. Los endpoints invocan directamente los métodos del dominio y traducen limpiamente las excepciones del negocio a códigos de estado HTTP estándar:

Código HTTP	Significado en TechStore	Ejemplo de Endpoint y Situación
200 OK	Operación exitosa o consulta completada.	POST /orders/O001/confirm -> Pasa a CONFIRMED.
404 Not Found	El pedido especificado no existe en memoria.	POST /orders/O999/confirm -> 'Pedido O999 no encontrado'
409 Conflict	Violación de transición en la máquina de estados.	POST /orders/O013/confirm (por segunda vez) -> 'Solo DRAFT'.

422 Unprocessable Entity	Violación de invariante o datos inválidos.	POST /orders/O007/confirm -> 'Requiere artículos y cliente activo'.
422 Unprocessable Entity	Límite de unidades excedido.	POST /orders/O009/items -> 'Máximo 20 unidades'.

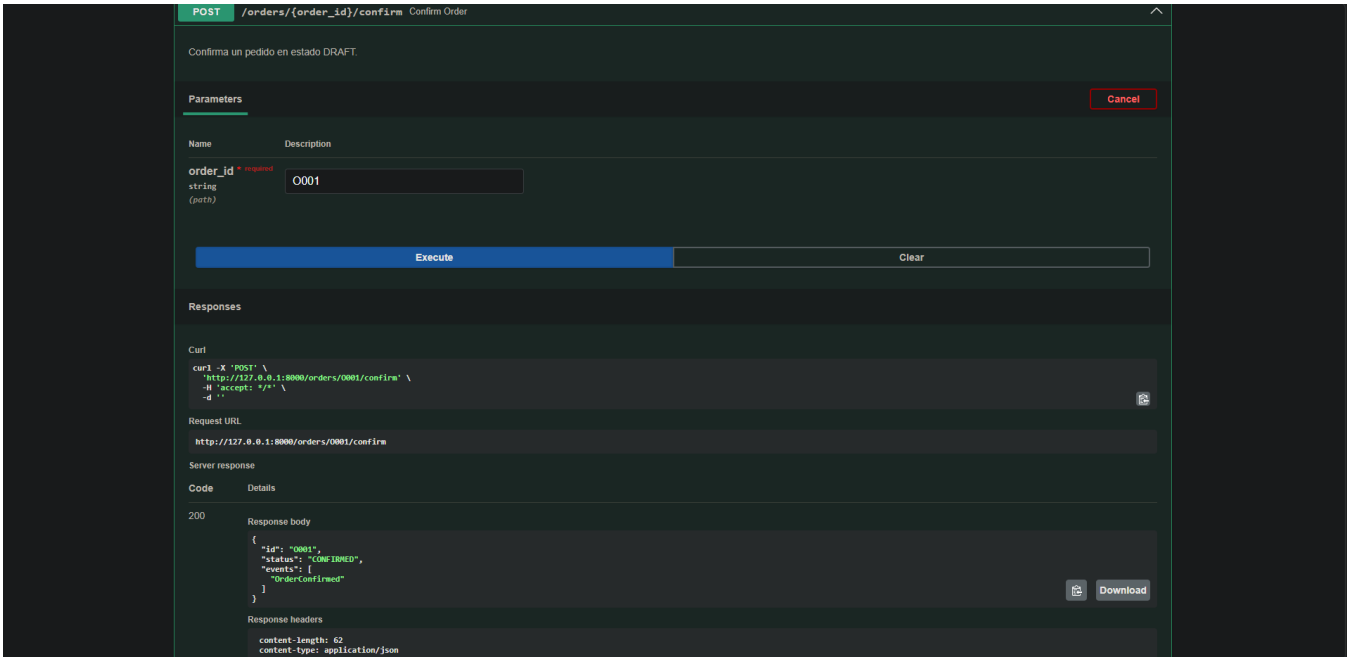


Figura 1. 200 OK al confirmar O001.

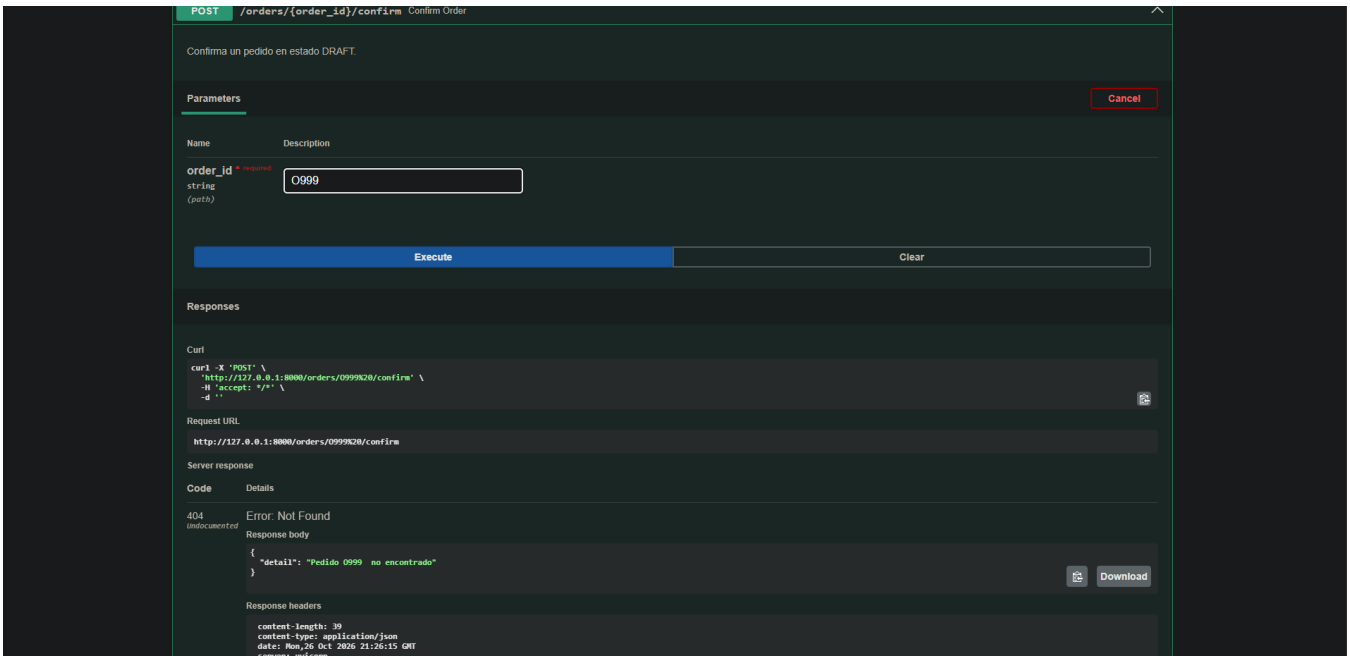


Figura 2. 404 Not Found con un pedido inexistente.

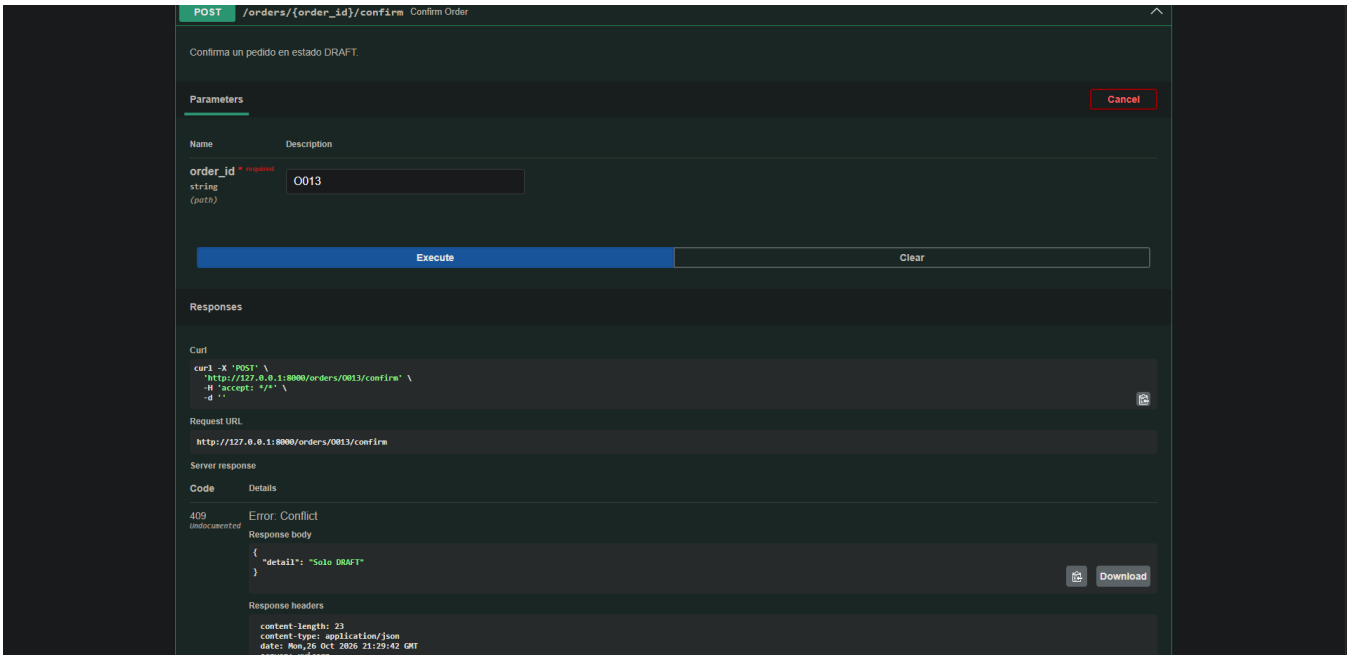


Figura 3. 409 Conflict al confirmar O013 por segunda vez.

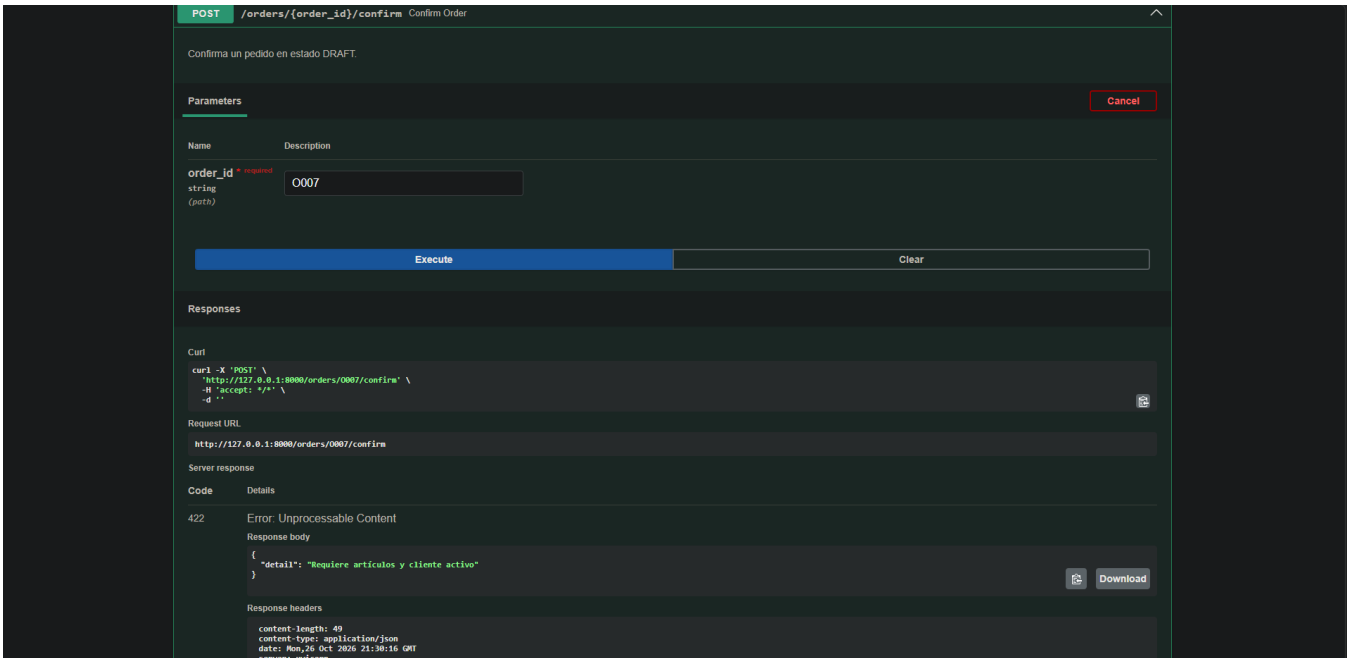


Figura 4. 422 al confirmar O007 (sin artículos).

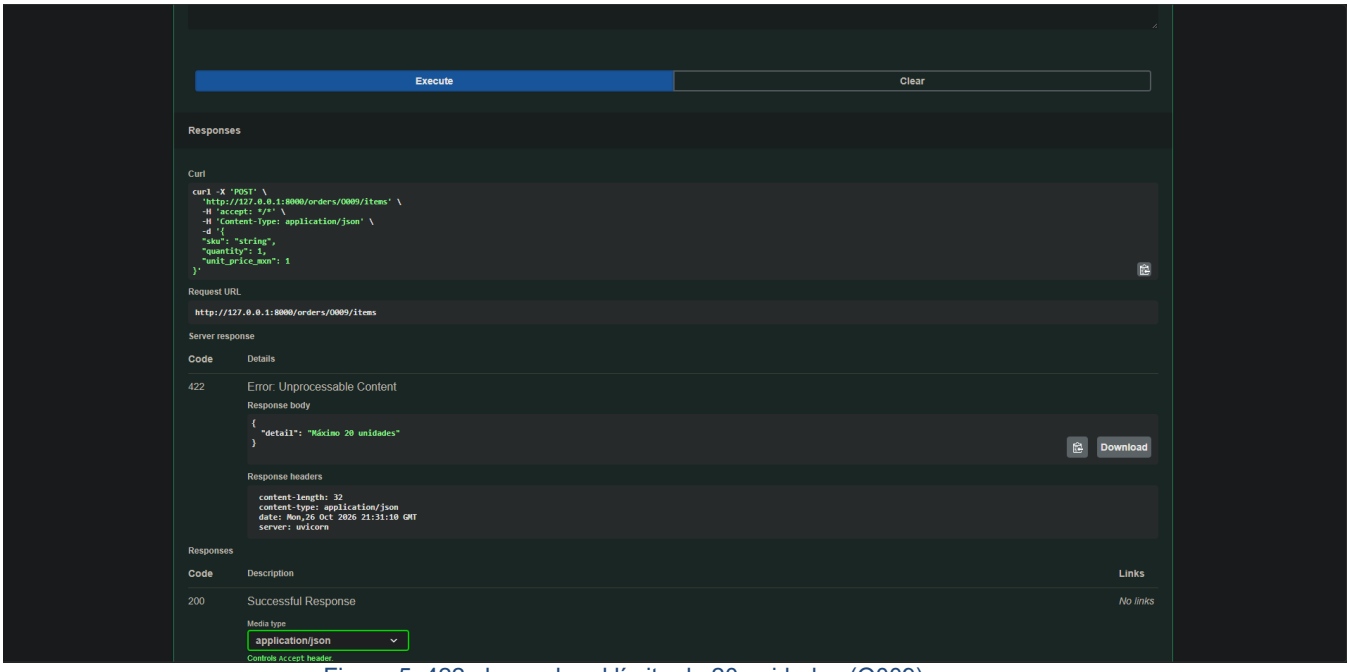


Figura 5. 422 al exceder el límite de 20 unidades (O009).

8. Pruebas Automáticas con Pytest

Se implementó una suite integral de 42 pruebas automatizadas distribuidas en tests/test_domain.py y tests/test_api.py, cubriendo los requerimientos y los escenarios de E01 a E15.

```
PS C:\Users\jaira\Documents\GitHub\Unidad 4> python -m pytest -v
plugins: anyio-4.15.1
collected 42 items

tests/test_api.py::test_api_root PASSED                                  [  2%]
tests/test_api.py::test_api_list_orders PASSED                           [  4%]
tests/test_api.py::test_api_get_order_valido PASSED                      [  7%]
tests/test_api.py::test_api_get_order_404_no_existe PASSED               [  9%]
tests/test_api.py::test_api_confirm_exitoso PASSED                       [ 11%]
tests/test_api.py::test_api_confirm_404_inexistente PASSED               [ 14%]
tests/test_api.py::test_api_confirm_409_doble_confirmacion PASSED        [ 16%]
tests/test_api.py::test_api_confirm_422_pedido_vacio PASSED              [ 19%]
tests/test_api.py::test_api_confirm_422_cliente_inactivo PASSED          [ 21%]
tests/test_api.py::test_api_ship_order PASSED                            [ 23%]
tests/test_api.py::test_api_ship_sin_confirmar_409 PASSED                [ 26%]
tests/test_api.py::test_api_cancel_order PASSED                          [ 28%]
tests/test_api.py::test_api_deliver_order PASSED                         [ 30%]
tests/test_api.py::test_api_return_order PASSED                          [ 33%]
tests/test_api.py::test_api_add_item_excede_limite_422 PASSED            [ 35%]
tests/test_api.py::test_api_evaluar_descuento_endpoint PASSED            [ 38%]
tests/test_domain.py::test_money_valido PASSED                           [ 40%]
tests/test_domain.py::test_money_negativo PASSED                         [ 42%]
tests/test_domain.py::test_money_suma_misma_moneda PASSED                [ 45%]
tests/test_domain.py::test_money_suma_monedas_distintas PASSED           [ 47%]
tests/test_domain.py::test_order_item_valido PASSED                      [ 50%]
tests/test_domain.py::test_order_item_cantidad_invalida PASSED           [ 52%]
tests/test_domain.py::test_order_item_sku_vacio PASSED                   [ 54%]
tests/test_domain.py::test_pedido_o001_inicia_draft PASSED               [ 57%]
tests/test_domain.py::test_limite_maximo_20_unidades_exito PASSED        [ 59%]
tests/test_domain.py::test_limite_maximo_20_unidades_excedido PASSED     [ 61%]
tests/test_domain.py::test_agregar_item_solo_en_draft PASSED             [ 64%]
tests/test_domain.py::test_confirmar_pedido_vacio_falla PASSED           [ 66%]
tests/test_domain.py::test_confirmar_cliente_inactivo_falla PASSED       [ 69%]
tests/test_domain.py::test_confirmar_exitoso_emite_evento PASSED         [ 71%]
tests/test_domain.py::test_confirmacion_doble_falla PASSED               [ 73%]
tests/test_domain.py::test_envio_sin_guia_falla PASSED                   [ 76%]
tests/test_domain.py::test_cancelacion_desde_shipped_falla PASSED        [ 78%]
tests/test_domain.py::test_flujo_completo_entrega PASSED                 [ 80%]
tests/test_domain.py::test_flujo_completo_devolucion PASSED              [ 83%]
tests/test_domain.py::test_descuento_10_porciento_vip PASSED             [ 85%]
tests/test_domain.py::test_descuento_0_porciento_no_vip PASSED           [ 88%]
tests/test_domain.py::test_descuento_bloqueado_por_promocion PASSED      [ 90%]
tests/test_domain.py::test_descuento_5_porciento_monto_superior_5000 PASSED [ 92%]
tests/test_domain.py::test_descuento_umbral_estricto_500 PASSED          [ 95%]
tests/test_domain.py::test_descuento_unidades_insuficientes PASSED       [ 97%]
tests/test_domain.py::test_escenarios_e01_a_e15_completos PASSED         [100%]
```

9. Tres Errores Encontrados, Solución y Aprendizaje

• Error 1: Imprecisión de tipos numéricos binarios (float vs Decimal)

Problema: Al calcular importes monetarios o comparar umbrales (>500.00 o >5000.00), el uso de float introduce errores de redondeo de punto flotante de base 2 (ej. 0.1 + 0.2 != 0.3).
Solución implementada: Se implementó el Value Object inmutable Money utilizando la clase Decimal de Python, asegurando que los montos se inicialicen desde cadenas numéricas exactas ('600.00') y validando invariantes (no negativos y valores finitos).
Aprendizaje: En sistemas financieros y de comercio electrónico, los tipos de coma flotante son inaceptables; el modelado con Value Objects inmutables garantiza exactitud y coherencia en cada operación contable.

• Error 2: Ambigüedad entre conteo de renglones y suma de unidades físicas

Problema: Existía el riesgo de evaluar el límite de 20 calculando len(order.items), lo cual contaría los tipos de productos y no las unidades físicas reales. **Solución implementada:** Se definió la propiedad @property units calculando explícitamente sum(item.quantity for item in self.items), y la validación en add_item() comprueba if self.units + item.quantity > 20 antes de agregar el artículo a la lista interna.
Aprendizaje: Las invariantes del dominio deben apegarse rigurosamente al significado del negocio (unidades físicas de almacén) y no a la estructura técnica de almacenamiento (renglones de detalle).

• Error 3: Emisión prematura de Domain Events ante fallos de validación

Problema: Si un evento de dominio se genera antes de completar todas las validaciones de una transición, un fallo posterior dejaría eventos huérfanos registrados en la orden sin que el estado haya cambiado.
Solución implementada: Se estableció el principio de tres fases: (1) Validar precondiciones e invariantes; (2) Modificar la variable de estado (status); (3) Instanciar y anexar el DomainEvent a la lista order.events. **Aprendizaje:** Los eventos de dominio representan hechos que efectivamente sucedieron en el negocio. Si una regla es rechazada, no hay hecho que registrar, garantizando la consistencia del log de auditoría.

10. Respuestas a las Preguntas del Reporte

1. ¿Por qué se usa Decimal en lugar de float para precios?

Porque los tipos float de punto flotante IEEE 754 representan números en base 2, lo que genera errores de representación en números decimales simples (como 0.1 o 0.05). En sistemas de cobro y comercio electrónico, estas pequeñas discrepancias se acumulan en cálculos de subtotales, impuestos y descuentos, provocando diferencias contables ilegales o fallas en auditorías. Decimal utiliza base 10 con aritmética exacta de precisión fija, garantizando que operaciones como sumas monetarias y porcentajes arrojen siempre resultados exactos al centavo.

2. ¿Cuál es la diferencia entre contar renglones y sumar unidades?

Contar renglones (len(order.items)) mide la diversidad de productos o artículos distintos (SKUs) agregados al pedido. En cambio, sumar unidades (sum(item.quantity for item in order.items)) calcula la cantidad total de piezas físicas que el almacén debe empacar. Si un pedido contiene 1 solo renglón con 20 unidades del producto SKU003, el conteo de renglones es 1, pero la suma de unidades es 20. La regla de negocio de TechStore limita el volumen físico del pedido (máximo 20 unidades), por lo que validar únicamente renglones permitiría pedidos de cientos de piezas, violando la invariante.

3. ¿Qué ventaja tiene encapsular una regla en Specification?

El patrón Specification desacopla por completo la lógica de evaluación del modelo de dominio y de los controladores. Sus principales ventajas son:
a) Reutilización: La misma regla puede usarse en la API, en el dominio o en reportes sin duplicar código.
b) Composición booleana: Permite construir reglas de negocio complejas y legibles combinando especificaciones simples mediante operadores (&, |, ~), facilitando cambios en las políticas de descuento sin tocar las entidades.
c) Testabilidad: Cada especificación se prueba de manera aislada con pruebas unitarias simples y directas.

4. ¿Por qué los eventos se registran solo después de una transición válida?

Un Domain Event (Evento de Dominio) representa un hecho inmutable que ocurrió en el pasado dentro del dominio ('OrderConfirmed', 'OrderShipped'). Si se registrara antes de validar o si la transición fallara por datos inválidos, el sistema registraría falsamente que ocurrió un evento que en realidad fue rechazado. Esto provocaría efectos secundarios indeseados (como notificar al cliente o generar una factura errónea) e invalidaría la consistencia y auditabilidad del historial del pedido.

5. ¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz?

Se podría reutilizar el 100% de la capa de dominio (app/domain.py: Money, Customer, OrderItem, Order, OrderStatus, DomainEvent) y de las reglas de negocio (app/specifications.py: todas las Specifications y evaluate_discount), así como los cargadores de datos (cargar_csv.py). Ninguno de estos módulos depende de FastAPI ni de Starlette. Podrían emplearse sin ningún cambio en una aplicación de consola CLI, una aplicación de escritorio (Tkinter/PyQt), una arquitectura de microservicios con gRPC o un framework web alternativo como Django o Flask.

11. Lista de Verificación Final

Antes de la entrega definitiva, se comprobó rigurosamente cada uno de los puntos estipulados en la guía de la práctica:

Ítem de Verificación	Criterio de Validación	Estatus
☑ Puedo ejecutar python cargar_csv.py sin errores.	COMPROBADO (Ejecución exitosa de los 16 pedidos)	**CUMPLIDO**
☑ O001 muestra 4 unidades y 2100.00 MXN.	COMPROBADO (O001 DRAFT 4 2100.00)	**CUMPLIDO**
☑ La regla del 10 % requiere VIP, >500, >=4 unidades y ausencia de promoción.	COMPROBADO (Verificado en E01 y tests)	**CUMPLIDO**
☑ La regla del 5 % requiere >5000 y ausencia de promoción.	COMPROBADO (Verificado en E05 y tests)	**CUMPLIDO**
☑ Una operación inválida no cambia el estado ni añade eventos.	COMPROBADO (Verificado en E03, E06, E11, E12, E13)	**CUMPLIDO**
☑ La API responde 404, 409 y 422 en los casos indicados.	COMPROBADO (Verificado en test_api.py y Swagger)	**CUMPLIDO**
☑ pytest termina sin fallos.	COMPROBADO (42 pruebas superadas al 100%)	**CUMPLIDO**
☑ El repositorio contiene README y evidencias.	COMPROBADO (README.md, .gitignore, código y reporte)	**CUMPLIDO**

Conclusiones:

La adopción de los principios del Diseño Orientado al Dominio (DDD) en el núcleo de TechStore demostró ser una estrategia altamente efectiva para aislar las reglas de negocio de la capa de infraestructura. Al modelar el sistema con entidades inmutables y de agregado, se garantizó que los detalles técnicos, como el uso de FastAPI o el almacenamiento de datos, no contaminaran la lógica central de la aplicación.

Un logro técnico fundamental fue la implementación del patrón Specification para la evaluación de descuentos, lo cual permitió componer reglas de negocio complejas mediante operadores booleanos de forma modular, clara y sin alterar la estructura del pedido. Complementariamente, el control del ciclo de vida del pedido mediante una máquina de estados determinista y la emisión de Eventos de Dominio aseguró la atomicidad del sistema. Al registrar estos eventos únicamente tras validar todas las invariantes y transiciones, se previno la creación de historiales inconsistentes.

Finalmente, la resolución de errores críticos de modelado reforzó la fiabilidad operativa del proyecto. El uso del tipo Decimal para evitar las imprecisiones de los cálculos de punto flotante en importes monetarios, junto con la correcta distinción entre renglones de detalle y unidades físicas, protegió rigurosamente las reglas contables y de inventario. El éxito de las 42 pruebas automatizadas confirma que se logró un diseño robusto y completamente desacoplado, listo para ser reutilizado en cualquier otro entorno o framework en el futuro.